

Sisteme de operare

Laborator 11

Semestrul I 2025-2026

Laborator 11

- advanced I/O
 - operatii scatter/gather
 - memory mapped files

Operatii scatter/gather

- permit citirea/scrierea din/in buffere necontigue intr-un singur apel
- scatter *read*/ gather *write*
- folosesc vectori de buffere

```
struct iovec {  
  
    void *iov_base; // adresa de inceput a bufferului  
  
    size_t iov_len; // dimensiunea bufferului  
  
}  
  
ssize_t readv(int fd, const struct iovec iov[], int iovcnt);  
  
ssize_t writev(int fd, const struct iovec iov[], int iovcnt);
```

Operatii scatter/gather (cont.)

ssize_t readv(int fd, const struct iovec iov[], int iovcnt);

- imprastie datele in buffere in ordine
- intotdeauna umple un buffer inainte sa treaca la urmatorul
- intoarce nr total de octeti cititi sau 0 daca nu mai sunt date (EOF)

ssize_t writev(int fd, const struct iovec iov[], int iovcnt);

- aduna datele de output din buffere in ordine *iov[0], iov[1], ... iov[iovcnt - 1]*
- intoarce nr total de octeti scrisi (in mod normal, suma dimensiunii bufferelor)

Memory-mapped files

- incarca continutul unui fisier intr-un buffer de memorie a.i. accesul la un element din buffer sa rezulte intr-un acces la disc
 - citirea unui element din buffer genereaza o citire din fisier
 - scrierea unui element din buffer rezulta intr-o scriere automata in fisier
 - astfel se executa I/O fara apelurile sistem *read/write*
- apel sistem care instruieste kernelul sa mapeze o regiune a unui fisier intr-o regiune din memoria RAM

```
caddr_t mmap(caddr_t addr, size_t len, int prot, int flag,  
              int fd, off_t off);
```

- *caddr_t* e in general *char **
- intoarce adresa zonei de memorie in care e mapat fisierul

Argumente mmap

- *addr* = adresa de inceput e zonei de memorie unde vrem sa incarcam continutul fisierului
 - in general, *addr* = 0 pt a lasa kernelul sa aleaga adresa
 - daca *addr* != 0, trebuie sa fie multiplu al paginii de memorie virtuala obtinuta cu *sysconf(_SC_PAGESIZE)*
- *fd* = descriptorul fisierului al carui continut se mapeaza in memorie
 - fisierul trebuie deschis in prealabil
- *len* = nr de octeti mapati
- *off* = offsetul de start in fisier de unde se mapeaza octeti in memorie
 - multiplu al paginii de memorie virtuala
- *prot* = PROT_READ, PROT_WRITE, PROT_EXEC, PROT_NONE
 - trebuie sa se potriveasca cu modul in care a fost deschis fisierul
 - ex: *mmap(..., PROT_WRITE, ..., fd, ...)* nu merge cu *open(fd, O_RDONLY)*

Argumente mmap (cont.)

- *flag*
 - MAP_FIXED: valoarea de retur trebuie sa fie egala cu *addr*
 - daca nu e specificat si *addr* != 0, kernelul considera *addr* o sugestie
 - pt portatibilitate maxima *addr* = 0
 - MAP_SHARED: afecteaza operatiile de scriere in zona de memorie
 - daca flagul e setat => scrierea in buffer rezulta automat intr-o scriere pe disc
 - altfel spus, alte procese care partajeaza fisierul vad modificarea
 - MAP_PRIVATE: scrierile in zona mapata rezulta in crearea unei copii a fisierului mapat; toate referintele ulterioare la elementele modificate sunt efectuate asupra copieii
 - orice apel *mmap* **trebuie** sa foloseasca fie MAP_SHARED fie MAP_PRIVATE
 - MAP_ANONYMOUS: zona de memorie nu are asociat un fisier
 - initializata cu 0
 - *fd* si *off* sunt ignorate
 - impreuna cu MAP_SHARED implementeaza o zona de memorie partajata intre procese inrudite

Observatii

- Q: pt *off* $\neq 0$, ce se intampla daca dimensiunea zonei de memorie mapate nu e multiplu al paginii sistemului?
 - ex: pp. dimensiunea fisierului e 96 octeti si pagina sistemului 4096
=> dupa *mmap* se obtine o regiune de memorie de 4096 de octeti, dintre care ultimii 4000 sunt setati pe 0
Obs: daca se scrie dincolo de offsetul 96, modificarile nu se reflecta in fisier
- semnale
 - SIGSEGV – generat pt acces la o zona de memorie la care nu avem permisiune de acces
 - generat si daca scriem intr-o zona de memorie RO
 - SIGBUS – generat de accese fara sens intr-o portiune a zonei de memorie mapate
 - ex: fisier mapat de un proces in intregime
 - alt proces trunchiaza fisierul inainte ca primul proces sa acceseze fisierul mapat
 - accesul la zona de memorie care a fost trunchiata rezulta in SIGBUS
- o regiune de memorie mapata pentru un fisier se mosteneste de catre copil dupa *fork*, evident nu si daca se apeleaza ulterior *exec* !

`munmap`

- zona de memorie se demapeaza:
 - implicit, la sfarsitul procesului (prin *exit*)
 - explicit cu *munmap*
 - inchiderea descriptorului de fisier nu demapeaza regiunea de memorie

```
int munmap(caddr_t addr, size_t len);
```

- nu afecteaza fisierul mapat (continutul ei nu e trimis pe disc)
- cu MAP_SHARED modificarile sunt automat scrise pe disc, nu cu *unmap* !

Concluzii mmap

- metoda mai rapida la copierea fisierelor (vizibil in cazul fisierelor mari)
- limitari: nu putem folosi *mmap* pentru conexiuni de retea sau pentru device-uri speciale (eg, terminal)
- e nevoie de atentie daca dimensiunea fisierului mapat se schimba dupa mapare
- in general simplifica algoritmi ori de cate ori manipularea unui buffer de memorie e preferabila operatiilor de I/O explicite (*read/write*)

Protectia accesului la memorie

- spatiul de adrese virtuale al procesului protejat de MMU, dar raman doua probleme:
 - protectia zonelor de memorie partajata
 - protectia accesului din program
- *mprotect* seteaza drepturile de acces la o zona de memorie

*int mprotect(void *addr, size_t len, int prot);*

- zona de memorie protejata: [*addr*, *addr* + *len*]
- *addr* trebuie aliniata la nivel de pagina
- *prot* (ca mai devreme) = PROT_READ, PROT_WRITE, PROT_EXEC, PROT_NONE
- return value: 0 succes, -1 eroare
- violarea drepturilor de acces rezulta in SIGSEGV (v. *mmap*)

Sisteme de operare

Laborator 11

Advanced I/O

1. Creati un nou subdirector *lab11/* in structura de directoare a laboratorului creata anterior (*SO/laborator*) si subdirectoarele aferente *doc src* si *bin*. Nu uitati sa actualizati variabila de mediu *PATH* pentru a include directorul *SO/laborator/lab11/bin*.

2. Scrieti un program **http-response.c** care emuleaza raspunsul unui server HTTP atunci cand primeste o cerere pentru un anumit document HTML. Programul primeste numele documentului HTTP solicitat de client in linie de comanda si afiseaza **pe ecran** un raspuns conform protocolului HTTP, dupa cum urmeaza.

In primul rand, conform protocolului HTTP, un server HTTP (deci si **http-response.c**) trebuie sa inceapa raspunsul prin a trimite un header text de forma urmatoare:

```
"HTTP/1.1 200 OK\n"
"Content-Type: text/html; charset=utf-8\n"
```

Programul **http-response.c** va stoca acest string intr-o variabila denumita *header*.

Apoi, serverul incearca sa identifice documentul (fisierul) solicitat de client. Daca fisierul nu exista, serverul trimite un raspuns de tipul

```
"HTTP/1.1 404 Not Found\n"
"Content-Type: text/html\n\n"
```

Cele doua newline-uri de la sfarsitul liniei *Content-Type* indica terminarea headerului (conventie a protocolului HTTP). Programul **http-response.c** va afisa acest raspuns pe ecran si se va incheia.

Daca documentul solicitat exista, serverul trebuie sa completeze ultima linie a headerului cu dimensiunea fisierului, pe care o ataseaza in header astfel:

```
"Content-Length: <size>\n\n"
```

unde in loc de *<size>* veti trece dimensiunea fisierului cerut de client si returnata de apelul sistem *stat/fstat*. Remarcati cele doua newline-uri de la sfarsitul liniei de mai sus care indica sfarsitul header-ului HTTP. Programul **http-response.c** va stoca acest string intr-o variabila denumita *content_length*.

Apoi serverul mapeaza *privat* (MAP_PRIVATE) fisierul solicitat in memorie cu ajutorul *mmap* si trimite continutul zonei de memorie aferente documentului catre client, dupa care inchide conexiunea. Programul **http-response.c** va stoca inceputul zonei de memorie reprezentand documentul intr-o variabila denumita *file_content*.

In acest moment, **http-response.c** este in masura sa emuleze functionarea serverului de HTTP tiparind pe ecran variabilele *header*, *content_length* si *file_content* pentru a forma un raspuns conform protocolului HTTP. Folositi *writew* pentru a realiza tiparirea celor trei string-uri pe ecran.

Indicatii: scrieti o functie ca mai jos pentru a implementa functionalitatea descrisa mai sus:

```
int http_response(char *filename, int fd);
```

Primul parametru reprezinta fisierul solicitat de client, al doilea reprezinta file descriptorul resursei unde se va trimite raspunsul (mai exact, in cazul **http-response.c**, *fd* va avea valoarea 1). Apoi, in *main*, programul va apela functia ca mai jos:

```
http_response(argv[1], 1);
```

Ca parametru al programului **http-response.c** puteti folosi pentru simplitate un fisier *index.html* care redirecteaza clientul catre adresa <https://fmi.unibuc.ro>, ca mai jos:

```
<head>
<meta http-equiv="Refresh" content="0; URL=https://fmi.unibuc.ro/" />
</head>
```

In acest fel, odata ce ati compilat programul **http-response.c** puteti sa-i redirectati outputul intr-un fisier html pe care sa-l incarcati apoi intr-un browser pentru a vedea daca raspunsul a fost bine format (i.e., browserul il intelege), ca mai jos:

```
$ http-response index.html > response.html
```

3. Modificati programul de mai sus si creati un program **multiply.c** care scrie cele trei componente ale raspunsului HTTP de mai sus de cate 100000 de ori pe ecran (evident, veti folosi redirectarea *stdout* pentru a recupera rezultatele). Folosind compilarea conditionata, veti scrie o varianta care foloseste la fiecare iteratie trei apeluri sistem *write* pentru a tipari **pe rand** *header*, *content_length* si *file_content*, respectiv o varianta care la fiecare iteratie foloseste *writew* ca in exercitiul precedent. Apoi comparati timpii de executie folosind comanda *time* ca mai jos:

```
$ gcc -o multiply multiply.c                # varianta cu 3 apeluri write
$ time multiply index.html > out
$ gcc -o multiply multiply.c -DWRITEV      # varianta cu writew
$ time multiply index.html > out
```

Ce observati? Care din variante este mai performanta si de ce? Ce dimensiune are fisierul *out*?

4. Scrieti un program **mmap-cp.c** care copiaza un fisier sursa intr-un fisier destinatie folosind apeluri sistem *mmap*. Numele fisierelor sunt furnizate in linie de comanda ca mai jos:

```
$ mmap-cp <fisier-sursa> <fisier-destinatie>
```

si programul le mapeaza in memorie cu ajutorul *mmap* inainte de a face copierea datelor.

Dupa ce ati testat corectitudinea programului, folositi un fisier de dimensiune relativ mare (eg, cca 100MB) drept fisier sursa si comparati performanta **mmap-cp** cu cea a programului **mycp** din laboratorul 2 folosind comanda *time* in felul urmator:

```
$ time mmap-cp fisier-100MB copie-mmap
$ time mycp fisier-100MB copie-mycp
```

Cum interpretati rezultatele comenzii *time*? Ce va spune breakdown-ul timpilor de executie?

Indicatie: pentru ca diferentele de timp de executie sa fie clare, aveti nevoie de un fisier sursa de dimensiune relativ mare (mai sus se sugereaza folosirea unui fisier de cca 100MB). Pentru a crea un asemenea fisier puteti folosi programul **multiply.c** de mai sus.

5. Scrieti o varianta **mmap-shm.c** a programului **shm.c** din laboratorul 9 in care zona de memorie partajata dintre cele doua procese se realizeaza cu *mmap* apelat cu parametrul `MAP_ANONYMOUS` (lucru posibil pentru ca avem de-a face cu procese inrudite, parinte-copil). Apoi folositi semafoare POSIX **fara nume** pentru a asigura sincronizarea dintre cele doua procese.

Indicatie: pentru a folosi semafoare POSIX fara nume e nevoie sa le plasati intr-o zona de memorie partajata pentru ca folositi procese si nu thread-uri. O posibila solutie este sa stocati cele doua semafoare la inceputul zonei de memorie mapate cu *mmap*.

6. Scrieti un program C **mprotect.c** care creeaza o zona de memorie partajata cu ajutorul *mmap* si `MAP_ANONYMOUS` intre doua procese parinte si copil. Procesul parinte creeaza zona de memorie partajata *read-only* si apoi creeaza procesul copil. Procesul copil doarme 2 secunde, apoi scrie un mesaj intai in zona de memorie partajata si apoi pe ecran, reseteaza cu *mprotect* drepturile de acces la zona de memorie partajata pentru a deveni *read-only* si termina executia. In tot acest timp parintele verifica in mod activ (*busy waiting*) daca s-a modificat continutul memoriei partajate si, cand acest lucru s-a intamplat, parintele citeste mesajul copilului din zona de memorie partajata si il afiseaza pe ecran. Apoi apeleaza *wait* pentru a recupera starea de terminare a procesului copil si demapeaza zona de memorie partajata.

Cum puteti sa faceti programul de mai sus sa functioneze?